ShorlyNot

Quantum-Inspired Cyber Threat Detection for Digital Signature Security

Smart India Hackathon (SIH 2026) · Problem Statement PS 26141 · Team Egreen Quanta

What is this? ShorlyNot is an end-to-end quantum cybersecurity architecture solving digital signature vulnerabilities against both quantum and classical attackers. It replaces classical digital signatures (RSA/ECDSA/Ed25519) with **teleportation-based Quantum Digital Signatures (ShorlyNot-QDS-T1)** and detects cyberattacks in real time using **non-machine-learning statistical thresholding (Q-STDF)** derived from Hoeffding's Inequality.

1. Exactly Two Products in this Repository

1. skeleton/ (The Invention & Framework): Standalone Python framework and FastAPI microservice (:8000). Executes real Qiskit Aer teleportation circuits, Pauli eigenstate encoding, Hoeffding threshold checks, and BB84 QKD.
2. bank/ (The Proof of Concept): Complete mock banking portal (:8080) and live SOC dashboard. Every fund transfer is quantum-signed; attacks trigger automated application-level isolation (Stages S0 to S4).

2. How the Quantum Digital Signature (QDS-T1) Works

Step	Protocol Action	Underlying Physics / Quantum Math
1. Prepare	Alice hashes payload and derives L check bits.	Encodes into Pauli eigenstates: $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$
2. Entangle	Alice & Bob share Bell pairs via quantum channel.	$|\Phi+\rangle = (|00\rangle + |11\rangle) / \text{sqrt}(2)$
3. Teleport	Alice performs Bell-basis measurement (BSM).	Yields 2 classical correction bits (m1, m2) per check bit
4. Protect	Correction syndromes encrypted via BB84 session key.	PQC / AES-256-GCM authenticated wrapper
5. Verify	Bob applies Pauli correction U(m1, m2) and measures.	U in {I, X, Z, XZ}. Measures in Alice's basis beta.
6. Bound	Bob computes empirical mismatch rate p_hat = errors / n.	Accepts if and only if p_hat <= tau (Hoeffding Bound)

3. Threat Detection Engine (Q-STDF) — 100% Non-ML Deterministic Ladder

Detection uses a strict, fully explainable mathematical rule ladder evaluated on every transaction:

Attack Vector	Detection Mechanism	Statistical / Protocol Guarantee	Bank Action
FORGERY	p_hat > tau	Random guesser mismatch ~50% >> tau (~21%)	Stage S2 (Block)
REPLAY	Nonce in replay window	Cryptographic uniqueness cache	Stage S3 (Reset)
IMPERSONATION	key_id != signer binding	Identity mismatch with PKI / key table	Stage S2 (Block)
CHANNEL	PQC tag fail / QBER > 11%	Eavesdropping causes state collapse / noise	Stage S4 (Lock)
UNAUTH_VERIFY	Verifier lacking entitlement	Access-control enforcement	Stage S3 (Reset)

4. Mathematical Formulation: Hoeffding Statistical Threshold

Acceptance threshold **tau = p0 + sqrt(ln(1/delta) / (2n))** where p0 is honest channel noise (2%), delta is the false-rejection budget (1%), and n is check count (64).

- Default baseline: **tau = 0.02 + sqrt(ln(100)/128) = 0.2097 (~21.0%)**.
- Honest transmission mismatch: ~0.00% to 2.00% (always accepted).
- Forgery attempt mismatch: ~50.00% (probability of bypassing threshold is < 1.8 × 10■■).

5. The North Star Fit Test (Reproduce in 2 Minutes)

1. Launch scripts/run_demo.bat (or scripts/run_demo.sh).
2. Login to Mock Bank at **http://127.0.0.1:8080** as Alice (alice / alice123).

3. Transfer ■5,000 to Bob — verified with Qiskit Aer quantum circuit and committed.
4. Open SOC console at **/soc** and click 'Trigger Forgery Attack'.
5. System catches mismatch (p_hat ~ 0.50 > 0.21) -> bank escalates to Stage S2.
6. Return to transfer page -> Bank refuses new transactions (Application Lockdown).

GitHub Repository: **https://github.com/Sansyuh06/yessit2026-Shorly_Not** | SIH 2026 PS 26141